

TUTORIAL

Sistema de Simulação e Registro de Atendimentos

Uso de Bibliotecas Padrão do Python

1. Objetivo da atividade

O objetivo desta atividade é ensinar o uso correto de **bibliotecas padrão do Python**, mostrando como elas facilitam o desenvolvimento de programas mais organizados, eficientes e profissionais.

Ao final do tutorial, o aluno será capaz de:

- Importar e utilizar bibliotecas padrão
 - Trabalhar com fila real usando `collections.deque`
 - Simular dados usando `random`
 - Registrar data e hora com `datetime`
 - Realizar cálculos com `math`
 - Interagir de forma simples com o sistema operacional usando `os`
-

2. Contexto do problema

Será desenvolvido um **sistema de atendimento**, no qual:

- Pessoas entram em uma fila
- O atendimento acontece por ordem de chegada
- Cada atendimento é registrado com data e hora
- Alguns dados são gerados automaticamente
- Estatísticas simples são calculadas

Esse tipo de sistema é comum em:

- Clínicas
 - Bancos
 - Atendimento ao público
 - Sistemas de senha
-

3. Importação das bibliotecas

Antes de escrever qualquer lógica, é necessário importar as bibliotecas que serão utilizadas.

O aluno deve iniciar o código com:

```
from collections import deque import random import datetime import math import os
```

Explicação

- `deque` : estrutura de dados própria para filas
 - `random` : geração de dados aleatórios
 - `datetime` : acesso à data e hora do sistema
 - `math` : funções matemáticas prontas
 - `os` : interação básica com o sistema operacional
-

4. Criando a fila de atendimento com deque

Agora será criada a fila principal do sistema.

```
fila_atendimento = deque()
```

Explicação

- `deque()` cria uma fila vazia
 - Diferente de listas, o `deque` é mais eficiente para remover e inserir elementos
-

5. Função para gerar clientes automaticamente (random)

Para simular a chegada de pessoas, será utilizada a biblioteca `random`.

```
def gerar_cliente(): numero = random.randint(1000, 9999) return  
f"Cliente_{numero}"
```

Explicação

- `randint(1000, 9999)` gera um número aleatório
 - O nome do cliente é criado automaticamente
 - Isso evita que o usuário tenha que digitar tudo manualmente
-

6. Função para adicionar cliente na fila

```
def adicionar_cliente(fila): cliente = gerar_cliente() fila.append(cliente)  
print(f"{cliente} entrou na fila.")
```

Explicação

- `append()` adiciona o cliente ao final da fila
 - A fila respeita a ordem de chegada
 - O nome é exibido para controle do usuário
-

7. Função para atender o próximo cliente (deque)

```
def atender_cliente(fila, historico): if len(fila) == 0: print("Fila vazia.")
return cliente = fila.popleft() horario = datetime.datetime.now()
historico.append(horario) print(f"{cliente} atendido em
{horario.strftime('%d/%m/%Y %H:%M:%S')}")
```

Explicação

- `popleft()` remove o primeiro da fila
 - `now()` captura data e hora atual
 - O atendimento é registrado no histórico
 - A data é formatada para leitura humana
-

8. Criando o histórico de atendimentos

```
historico_atendimentos = []
```

Explicação

- Lista usada para armazenar os horários dos atendimentos
 - Esses dados serão usados para cálculos posteriores
-

9. Utilizando math para cálculos simples

```
def calcular_media_atendimentos(historico): total = len(historico) if total ==
0: return 0 return math.ceil(total)
```

Explicação

- `len()` conta atendimentos
 - `math.ceil()` arredonda para cima
 - Mostra o uso prático da biblioteca `math`
-

10. Limpando a tela com os

```
def limpar_tela(): os.system('cls' if os.name == 'nt' else 'clear')
```

Explicação

- Verifica o sistema operacional
- Limpa a tela para melhorar a visualização do sistema

11. Criando o menu do sistema

```
def menu(): print("1 - Adicionar cliente") print("2 - Atender cliente")  
print("3 - Ver total de atendimentos") print("0 - Sair")
```

12. Estrutura principal do programa

```
while True: menu() opcao = input("Escolha uma opção: ") if opcao == '1':  
adicionar_cliente(fila_atendimento) elif opcao == '2':  
atender_cliente(fila_atendimento, historico_atendimentos) elif opcao == '3':  
total = calcular_media_atendimentos(historico_atendimentos) print(f"Total de  
atendimentos: {total}") elif opcao == '0': print("Encerrando sistema.") break  
else: print("Opção inválida.")
```

Explicação

- Estrutura de repetição mantém o sistema ativo
- Cada opção chama uma função específica
- O programa só encerra quando o usuário escolhe sair